Just in the test runner itself, which we saw earlier in the screenshot, clicking on any of the command steps in the left-hand column displays the result of that step in the console where you can examine it more closely. Similarly, clicking on error messages on the left side will put the error stack trace in the console.

The cy.log command takes a string message and an arbitrary object. When invoked, it prints the message and the object to the left-side command rundown of the test runner, and clicking on that message puts the arbitrary object into the console. This can be a helpful way to log messages in tests and have them be displayed in the context of the code that displays the message.

Also, right above the actual browser display is a text bit that starts with cy.get or something similar. This is a little UI for interacting with the browser on display. You can enter a Cypress finder method at the top, see the result of that selector highlighted in the document, and print it to the console.

Using the React DevTools Extension

A few React-specific troubleshooting tools are worth becoming familiar with too. The most important is a browser extension called React DevTools. You can download this extension from the extension store of your favorite browser, and it looks something like the screen shownhere (this is the Chrome version).